

TestPlatform

Rapport d'Execution de Campagne

Campagne #5

Projet: Projet #11

Version: N/A

Environnement: Environnement #7

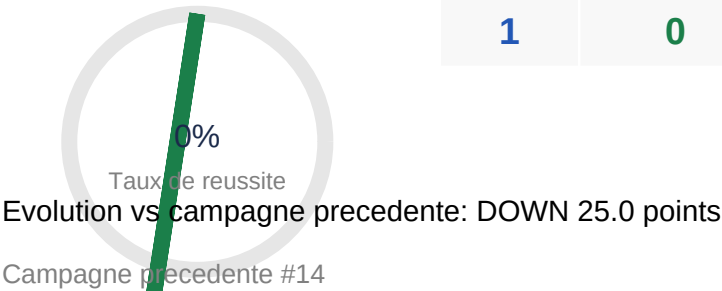
Date: 20/05/2026 16:46

ECHEC

Resume executif

Taux de reussite

Total	Succes	Echecs	Erreurs	Duree
1	0	0	1	49.2 s



Informations du contexte

Projet	Projet #11
Description	N/A

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	https://github.com/ilyeschaabani/sopport-pro-	main

Cas de test

Titre	Type	Priorite	Risque	Script
Test #21	UNIT	N/A	N/A	IA

Environnement

URL Web	N/A
URL API	N/A
Variables	N/A

Resultats details

Test #21

ERROR

Type	UNIT	Duree	7 ms
Script	IA	Erreur	IllegalStateException : workDir does not exist: \\tmp\\ms-execution\\exec-10120610124546855687\\Backend\\AuthenticationMicroService

Analyse IA

Test #21

Logs (extrait)	N/A
Analyse IA	La cause probable de l'échec du test est liée à une erreur d'instanciation de la classe `AuthenticationServiceImpl`. Spécifiquement, le champ `RoleRepository` a été transmis comme null dans son constructeur et ne peut donc pas être utilisé par le test. La correction à apporter au script est d'ajuster la construction de l'objet `authService` pour qu'il n'accepte pas un argument Null pour le `RoleRepository`, comme suit: <pre>```java authService = new AuthenticationServiceImpl(userRepository, null, authenticationManager, jwtService); ```</pre> Puisque nous ne souhaitons pas tester la persistance des rôles dans cet état du test (donc le `RoleRepository` est considéré comme non important pour l'application), on peut aussi simplement mettre un argument null au constructeur de `AuthenticationServiceImpl`. Si les utilisateurs sont autorisés uniquement à avoir des rôles et que ces informations ne doivent pas être testées, il n'y a rien à améliorer dans le script de test ou l'implémentation de `AuthenticationServiceImpl`.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
Test #21	Suite UNIT	7 ms

Temps d'execution par type

Type	Duree moyenne
UNIT	7 ms

Tests instables

Aucun test instable detecte

Recommandations

- Test #21: La cause probable de l'échec du test est liée à une erreur d'instanciation de la classe ``AuthenticationServiceImpl``. Spécifiquement, le champ ``RoleRepository`` a été transmis comme null dans son constr...

Annexe - Scripts generes par IA

Test #21

```
package org.example.authenticationmicroservice.testgen;

import org.example.authenticationmicroservice.Entity.User;
import org.example.authenticationmicroservice.Repository.UserRepository;
import org.example.authenticationmicroservice.Service.Authentication.AuthenticationServiceImpl;
import org.example.authenticationmicroservice.Service.JWT.JWTService;
import org.example.authenticationmicroservice.dto.JwtAuthenticationResponse;
import org.example.authenticationmicroservice.dto.SignInRequest;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.doThrow;
import static org.testng.Assert.*;

public class Generated_20_Test {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @Mock
    private AuthenticationManager authenticationManager;

    @Mock
    private JWTService jwtService;

    // Si d'autres dépendances sont nécessaires (RoleRepository, EntityManager),
    // il faudra les mocker également ou les passer dans le constructeur.
    // Pour simplifier, on suppose qu'on peut les mocker ici.

    private AuthenticationServiceImpl authService;

    @BeforeMethod
    public void setUp() {
        MockitoAnnotations.openMocks(this);
        // Adapter le constructeur selon ton service réel
        // Ici on suppose qu'il ne prend que ces 4 dépendances
        authService = new AuthenticationServiceImpl(
            userRepository,
            null, // RoleRepository si nécessaire
            passwordEncoder,
            authenticationManager,
            jwtService
        );
    }

    @Test
    public void testLoginSuccess() {
        // Given
        SignInRequest request = new SignInRequest();
        request.setEmail("test@example.com");
```

```
request.setPassword("password");

User mockUser = new User();
mockUser.setId(1L);
mockUser.setEmail("test@example.com");

given(userRepository.findByEmailWithRoles("test@example.com"))
    .willReturn(Optional.of(mockUser));
given(jwtService.generateToken(mockUser)).willReturn("fake-jwt-token");
given(jwtService.generateRefreshToken(any(), any())).willReturn("fake-refresh-token");

// When
JwtAuthenticationResponse response = authService.login(request);

// Then
assertNotNull(response);
assertEquals(response.getToken(), "fake-jwt-token");
}

@Test
public void testLoginFailureWrongPassword() {
    // Given
    SignInRequest request = new SignInRequest();
    request.setEmail("test@example.com");
    request.setPassword("wrong");

    doThrow(new BadCredentialsException("Bad credentials"))
        .when(authenticationManager).authenticate(any(UsernamePasswordAuthenticationToken.class));

    // When & Then
    assertThrows(BadCredentialsException.class, () -> authService.login(request));
}
}
```